Complex Task Allocation Problem

Ahmed Mahfouz*, Hazem Mahdy*, Loaie Solyman*, Mohamed Shelkamy*,*
*Mechatronics Department - Faculty of Engineering and Materials Science - German University in Cairo, Egypt
Email: ahmed.alimahfouz@student.guc.edu.eg, mohamed.shelkamy@student.guc.edu.eg,

I. State of Art

Paper [1] presents a Modified Distributed Bees Algorithm (MDBA) to solve MRTA problems that are developed to allocate stationary heterogeneous sensors to upcoming unknown tasks using a decentralized, swarm intelligence approach to minimize the detection times. Sensors are optimally allocated to tasks based upon sensors' performance, tasks' priorities, and the distances of the sensors from the locations where the tasks are executed. The algorithm was compared to a (DBA), a Bees System, and two common multi-sensor algorithms, which were fitted for the specific task.

In paper [2], the optimization problem of the wireless sensor network based on differential evolution is tackled. The methodology used in this work focuses on solving the minimum energy consumption in the path optimization model using the differential evolution algorithm. To verify the suggested strategy, a comparison was carried out using three approaches the classical algorithm, particle swarm optimization, and standard differential evolution algorithm. Finally, the results validated that using a differential evolution algorithm merited superior results in performance.

aper [3], aims at enhancing the performance of particle swarm optimization (PSO) via remedying two drawbacks on MRTA problem, this paper proposes an improved PSO method, iteration on number of PSO is presented, so the main control parameters of particles in the proposed PSO are adaptively adjusted. Since the convergence of particle swarm optimization remains paramount and dramatically affects the performance of PSO, this paper also analytically investigates the convergence of the proposed PSO. Paper [4] proposes a multi-robot planner inspired using ant colony optimization (ACO) to solve the MRTA problem. This planner searches to find collision-free paths to all tasks to be executed using a spread of ants from each robot. Neglecting the ones with collisions from other ants in their determined paths. This planner, which is called ACO-based Multi-Robot Planner is presented in this paper and has been tested against other similar planners producing promising results.

II. Simulated annealing technique

Simulated annealing algorithm depends on a number of operations that represents the algorithm efficiency such as cooling operation(we used a linear cooling method) and searching for the current solution neighbours so our SA algorithm we have used three diffrent operations to find the neighbours to create a variety in our algorithm and prevent it from fallen into local optimum so this is our three searching in the neighbours operations.

swap: This method swaps any two random tasks in the task array.

invert: This method chooses any two random tasks and invert the tasks sequence between them.

insert:this method choose any random task and insert it in a random position in our task list.

fitness: This method calculates the fitness value of the current solution. It calculates the total distance between the robots and their assigned tasks. It also calculates the total values of the robots utilities. The distance and utility values are constrained as the task can not be assigned to a robot if its energy level, or its utility does not allow it cover the distance toward the task or to perform the task respectively. Finally substitute the calculated values in the fitness function described by the following equation:

$$[H]f = w_1 \cdot \frac{1}{d_{total}} + w_2 \cdot utility_{total} \tag{1}$$

where w_1 and w_2 are the weight of the distance and the utility values respectively. for our encoding method we have used an array this array will represent our solution as the index will represent the corresponding task and the number inside the index will represents the corresponding robot.we also used another representation for our problem as to make it simpler we used a clustering method by distribute the tasks on the robots in a way hat cut the minimum distance and its utility is suitable for the corresponding robot so in this case our solution was a lists with each list represents the sequence of the tasks for each robot.

III. Genetic Algorithm

A. Genetic pseudo code

The Genetic algorithm was implemented based on several steps that are illustrated in the following pseudo code: our encoding princesses we have two arrays the first one indicated the tasks available in the system and the second one indicates the corresponding robots for the tasks for example if list1=[0,1,2,3] and list2=[0,0,1,1] this is mean that robot 1 will execute task 1 then task 2 and robot 2 will execute task 3 then task 4.

B. Results and Discussion

The algorithm was run on several tests and the results of some of these tests are shown in the following table and figures:

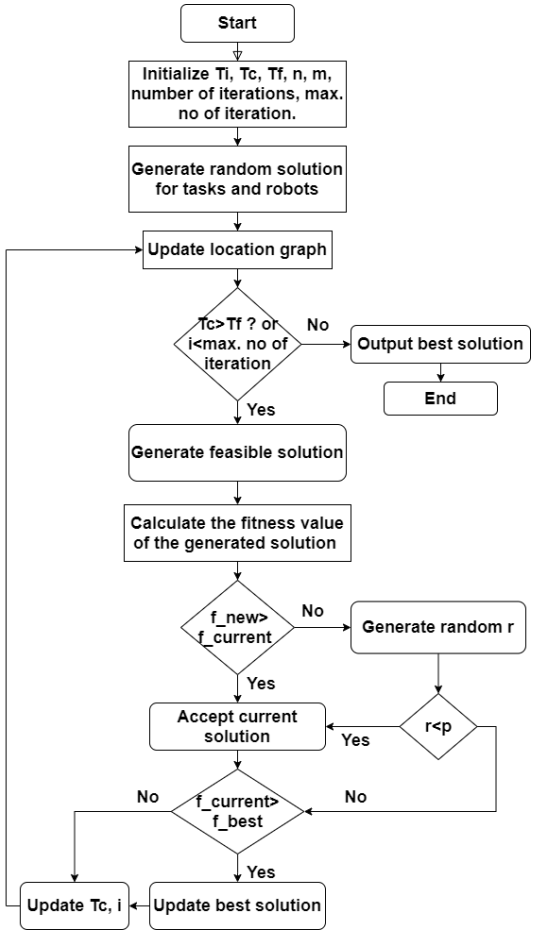

Fig. 1: Simulated Annealing Flow Chart

rob	tsk	best fitness value	average fitness	worst fitness value
2	10	0.000215	0.000214	0.000209
2	15	0.000141	0.000137	0.000129
3	16	0.000153	0.000151	0.000150
4	16	0.000168	0.000166	0.000161
3	20	0.000102	0.000099	0.000089
4	25	0.000078	0.000082	0.000091
10	10	0.955	0.955	0.955

TABLE I: Fitness value corresponding to a certain combination of robots and tasks from Genetic Algorithm

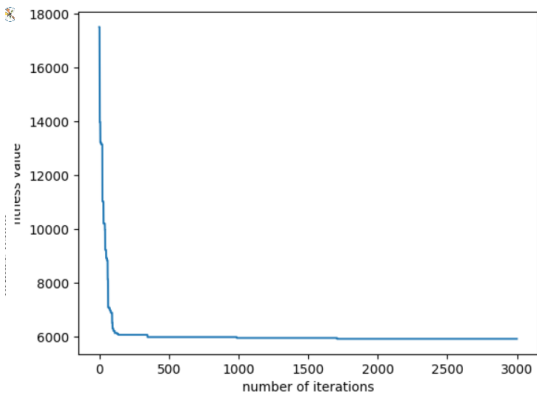

Fig. 2: The fitness curve in case in 4 robots and 16 tasks

Algorithm 1 Genetic Algorithm

0: **step 0**: Calculate distance matrix between robots and tasks, the tasks themselves and the utility matrix of robots with respect to tasks.

0: **step 1**: Gen:=1, generate the initial population for the tasks and their assigned robots.

0: **step 2**: if gen $>$ maxgen (algorithm iterations number), then turn to step 11, otherwise turn to step 3;

0: **step 3**: Calculate the fitness of every individual of parent generation and update best local fitness and global fitness reached so far (by calculating total travelled distance by vehicles and total utility), i:=1

0: **step 4**: if i>popsize, then turn to step 5, else turn to step 6;

0: **step 5**: Gen:=gen+1, competing between current parent generation and offspring, generate new parent generation;

0: **step 6**: Save the defined percentage of elite chromosomes of the this population in the new generation.

0: **step 7**: According to roulette wheel strategy, select two individuals p1 and p2 from parent generation, then randomly produce a real number P$\in [0, 1]$. if $P \leq P_c$, then turn to step 8; else turn to step 9;

0: **step 8**: Perform ordered cross over on the parents that represent the tasks and uniform cross over on the parents that represent the robots assigned to the these tasks. ;

0: **step 9**: Perform swap mutation over on the parents that represent the tasks and uniform mutation on the parents that represent the robots assigned to the these tasks.;

0: **step 10**: According to (7), calculates the fitness of o1 and o2, and insert them to offspring, i:=i+2 , turn to step 4;

0: **step 11**: Output the individual with the largest fitness, algorithm is ended. =0

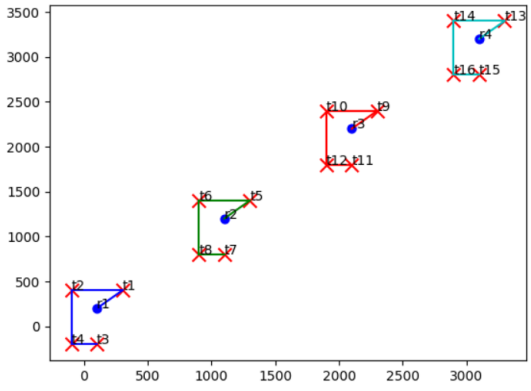

Fig. 3: Visualization of 4 robots and 16 tasks

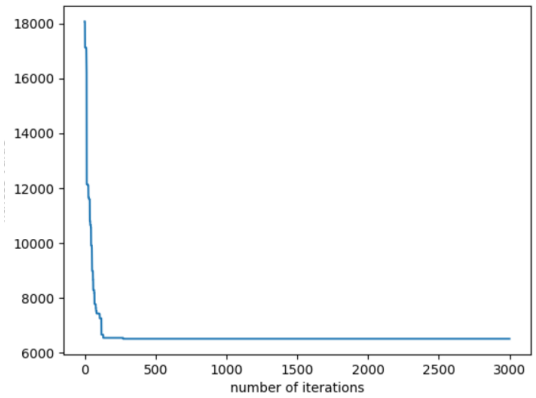

Fig. 4: The fitness curve in case in 3 robots and 16 tasks

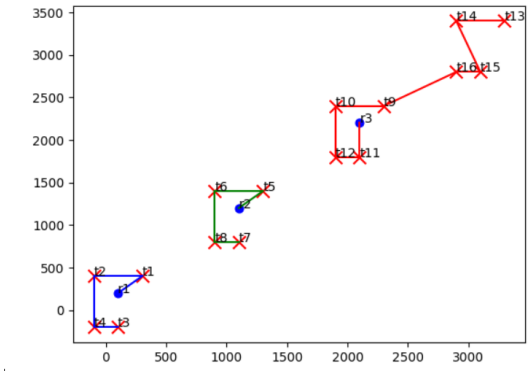

Fig. 5: Visualization of 3 robots and 16 tasks

we could notice that by in small number of tasks the algorithm could rapidly find the optimal or sub optimal solution but by increasing the number of robots and the tasks the algorithm become more sluggish to find the optimal or sub optimal solution for example when we increased the number of robots to 4 and tasks to 25 we cloud notice how sluggishly the algorithm get the sub optimal solution and the difference between the average and the best solution increased but the solution still good in comparison with the benchmark [5] as To validate the effectiveness of the algorithm implementation

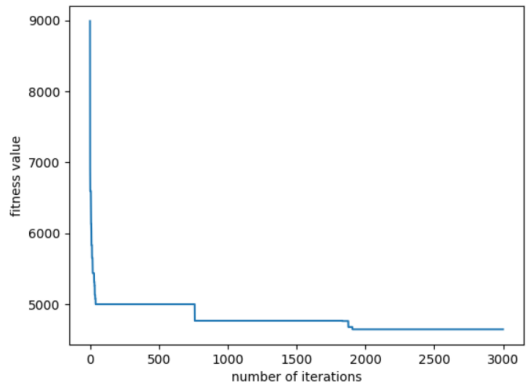

Fig. 6: The fitness curve in case in 2 robots and 10 tasks

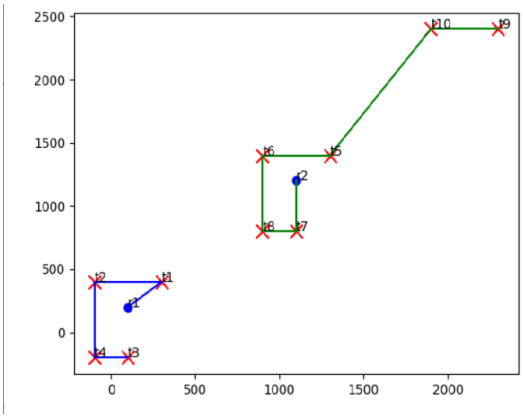

Fig. 7: Visualization of 2 robots and 10 tasks

a utility matrix was obtained from paper [5]. The test was run on using this matrix and the result is shown in the following figure as the algorithm succeeded to reach the optimal solution reached by the benchmark in case each robot assigned to only one task and it overcomes the results of the benchmarks when there is no constrained that each robot has to be assigned to only one task:

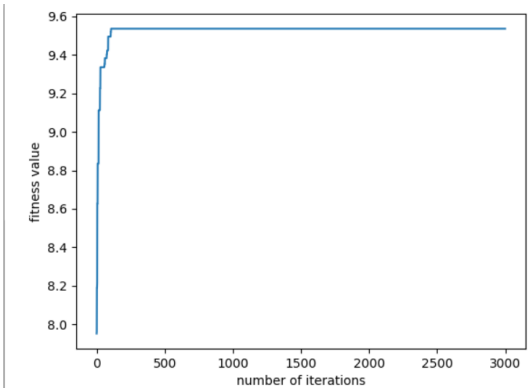

Fig. 8: The fitness curve in case in 10 robots and 10 tasks using the utility matrix

and for the comparison with SA algorithm we could notice that both algorithms showed a strong search capability to find the optimal or the sub optimal solution as the genetic algorithm overcome SA when SA annealing do not use the clustering method but when SA used with clustering it showed a rapidly search capability and a strong exploration and exploration capability but we could not compare between the GA till we uses clustering method with GA also but overall the 2 algorithm are suitable to solve MRTA problem.

IV. Ant Colony Algorithm

Algorithm 2 Ant-Colony (ACO) algorithm

$CostMatrix, evaporatingRate, InitialPheromone$
$terminationCondition, antsNumber, robots, tasks$
while *not terminationCondition* **do**
  while *number of used ants>0* **do**
    $set\ all\ tasks\ as\ unexcused$
    while *number of unfinished tasks>0* **do**
      calculate the probability to choose the next task
      if *energy level of the robot can not execute another task* **then**
        return to the depot
      else
        add the selected task into the route or return to the depot(according to the roulette wheel algorithm)
      end
    end
    $calculate\ the\ fitness function$
    $update\ the\ best\ solution$
  end
  evaporate pheromone & Deposition(locally and globally)
end

Pheromone updating: there are three different versions of ant system (AS) Ant-Density, Ant-Quantity and Ant-Cycle. An-t Density and Ant-Quantity updating pheromone is performed after selecting each edge. However, Ant-Cycle updating pheromone after all ants choose their edges. The third approach results were better than the first two approaches [**?**], so we will use the third approach to update pheromone. The process works as mentioned after artificial k ants has constructed a feasible solution, the pheromone trails are laid depending on the objective value L(k)(fitness function). For each arc [i, j] that was used by ant k, the pheromone trail is increased by L(k). In addition to that, all arcs belonging to the so far best(choose number of best solution to be globally pheromone update) solutions (objective value M) are emphasized as if o ants, so-called elitist ants had used these routes so these solution update their pheromone both locally and globally.

Pheromone evaporation: this process is responsible for getting ride of unlimited increase in pheromone trail. This process help the algorithm to forget the low quality solution as by the time the pheromone on the paths with low quality will approximately equal to zero. Evaporation process eliminates the value of pheromone by a predefined amount between 0 and 1.

A. Path Planning

Another test was performed on a constructed map including some obstacles. The Dijkstra algorithm was used to find the path of each robot after the tasks were distributed on the robots.

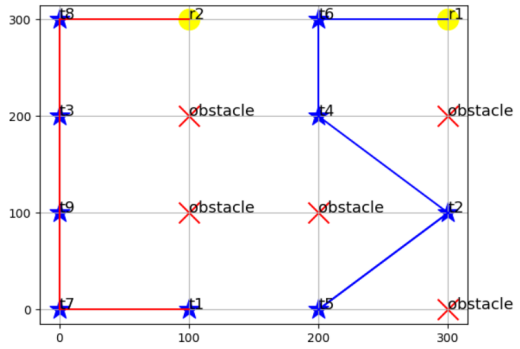

Fig. 9: Visualization of 2 robots and 9 tasks

Algorithm 3 Dijkstra

0: **step 1**: Create a set sptSet (shortest path tree set) that keeps track of vertices included in shortest path tree, i.e., whose minimum distance from source is calculated and finalized. Initially, this set is empty.
0: **step 2**: Assign a distance value to all vertices in the input graph. Initialize all distance values as INFINITE. Assign distance value as 0 for the source vertex so that it is picked first.
0: **step 3**:
0: **while** sptSet doesn't include all vertices **do**
0: a) Pick a vertex u which is not there in sptSet and has a minimum distance value.
0: b) Include u to sptSet.
0: c) Update distance value of all adjacent vertices of u. To update the distance values, iterate through all adjacent vertices. For every adjacent vertex v, if sum of distance value of u (from source) and weight of edge u-v, is less than the distance value of v, then update the distance value of v.
0: **end while**=0

But in addition to that, there is an array to store the path distances from the start point. As a result, if there is a shorter path, the array will save it. The result of this test is shown in the following figure:

B. Results and Discussion

The algorithm was run on several tests and the results of some of these tests are shown in the following table and figures:

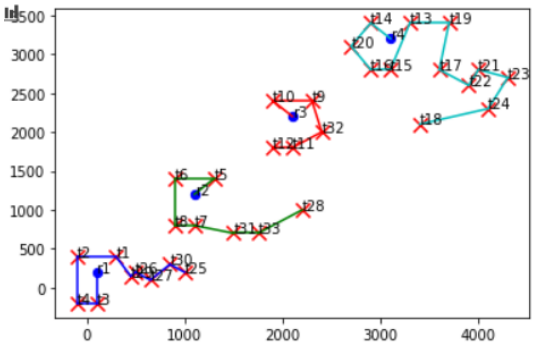

Fig. 10: Visualization of 4 robots and 35 tasks

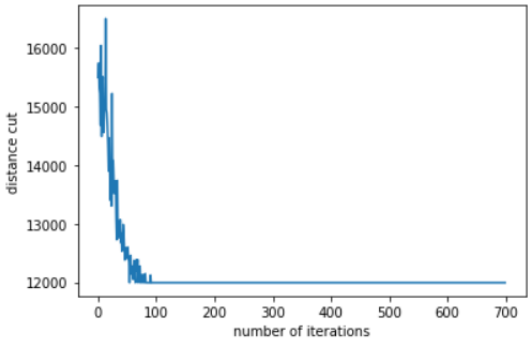

Fig. 11: The fitness curve in case in 4 robots and 35 tasks

robots	tasks	best	average	worst
2	10	0.000215	0.000215	0.000215
2	15	0.0001436	.0001436	.0001436
3	16	0.000153	.000153	.000153
4	16	0.000168	0.000168	0.000168
3	20	0.000119	0.000119	0.000119
4	25	0.0001028	0.0001028	0.0001028
4	35	0.000084	0.000084	0.000084
10	10	0.955	0.955	0.955
3	60	0.00004	0.000045	0.000049
2	60	0.00004	0.000045	0.000049

TABLE II: Fitness value corresponding to a certain combination of robots and tasks from ACO Algorithm

It is clear from the previous results that the algorithm could find the optimal or sub optimal solution for almost all the tests. For small number of tasks the best, average and worst solution were the same as the algorithm able to find repeatable solutions during several runs and the fitness rabidly converges. However, for large number of tasks the the best, average and worst solutions showed a slight difference and the fitness takes more time to converge.

A utility matrix was obtained from paper [5] to validate the algorithm implementation. The test was run on using this matrix and the result is the same as the one obtained by the Genetic algorithm. The algorithm succeeded to reach the optimal solution reached by the benchmark in case each robot assigned to only one task and it overcomes the results of the benchmarks when there is no constrained that each robot has to be assigned to only one task. Moreover, it was run on another

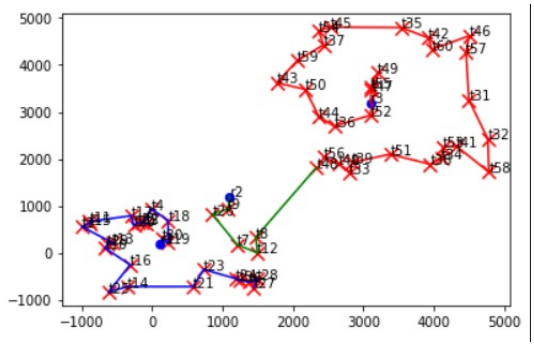

Fig. 12: Visualization of 3 robots and 60 tasks

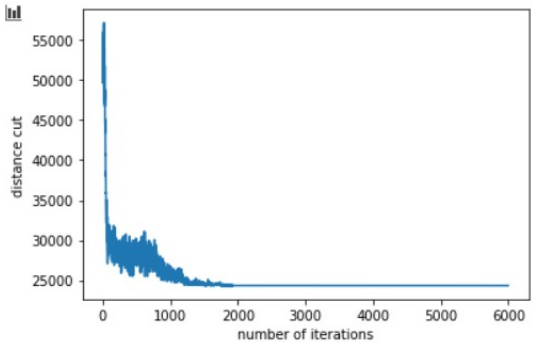

Fig. 13: The fitness curve in case in 3 robots and 60 tasks

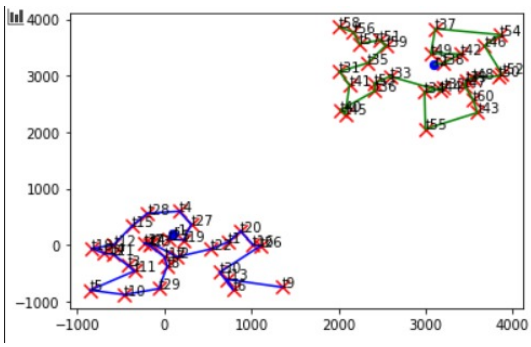

Fig. 14: Visualization of 2 robots and 60 tasks

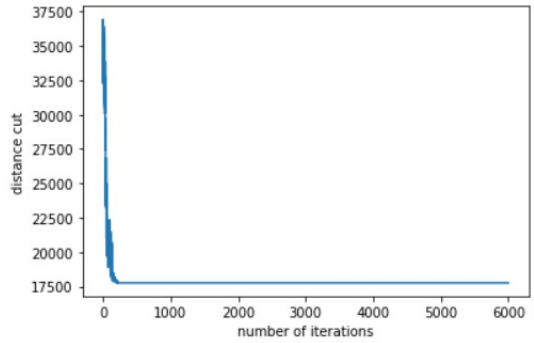

Fig. 15: The fitness curve in case in 2 robots and 60 tasks

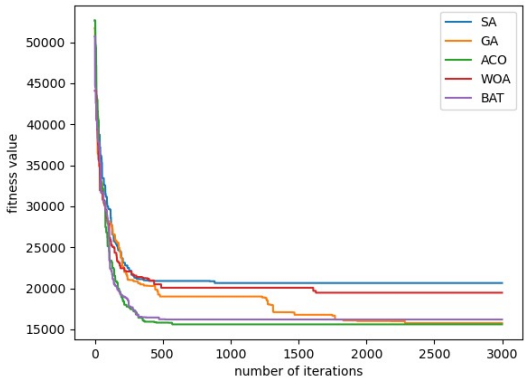

Fig. 16: The fitness curve in case in 5 robots and 35 tasks

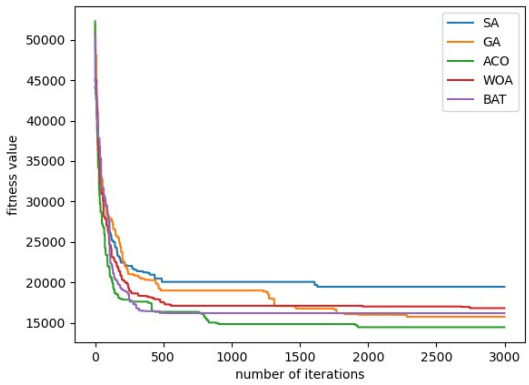

Fig. 17: The fitness curve in case in 3 robots and 33 tasks

benchmark that was mentioned in [6] which has one depot. The result was the same sequence in the paper which is (124 123 120 126 125 118 119 114 144 143 117 115 92 93 95 94 89 91 90 83 116 110 i 11 87 109 113 103 107 108 112 121 122 133 134 136 128 132 127 130 41 47 40 42 104 106 99 100 105 102 85 86 88 84 78 81 75 77'74 76 73 2 70 72 69 82 80 79 68 71 63 62 66 64 65 67 23 24 31 32 37 35 36 39 34 20 18 38 28 33 29 30 26 22 27 25 7 9 5 21 17 16 19 12 10 1 348 14 11 6 96 97 101 98 15 13 43 44 51 50 46 54 49 48 52 53 45 57 61 56 60 55 59 58 142 141 140 137 129 131 135 139 138) and the length is 30353.8609965236.

Task allocation is a very important problem in multi-robot cooperative research. This paper represents two of most commonly used stochastic approach in solving MRTA problem (Genetic Algorithm (GA) and Ant-Colony Optimization (ACO) Algorithm). Both show a strong space discovering for small number of tasks less than 50 task. In addition, both algorithms conclude that they have a strong space search capability, so that they can make the robot map reach the task fast, and obtain the global optimal scheme for the task allocation problem. However, with increasing the number of robots in each depots, the GA become more sluggish to find the optimal or sub-optimal solution compared to the ACO.as the exploration rate for both genetic and ACO is much more than SA as both genetic and ACO are population based as they search for the solution using several members but on the other hand SA search for the solution using only one member which make the exploration rate is low which make the SA algorithm as a slow algorithm so it takes much more time to find the optimal or sub optimal solution.this is show from our results.we could see also ACO is the most suitable algorithm to solve multi depot multi robot task allocation.

V. Whale Algorithm

VI. Conclusion

Task allocation is a very important problem in multi-robot cooperative research. This paper represents 5 of most commonly used stochastic approach in solving MRTA problem (Genetic Algorithm (GA) and Ant-Colony Optimization (ACO) Algorithm,SA,DWOA).the 4 proposed algorithms

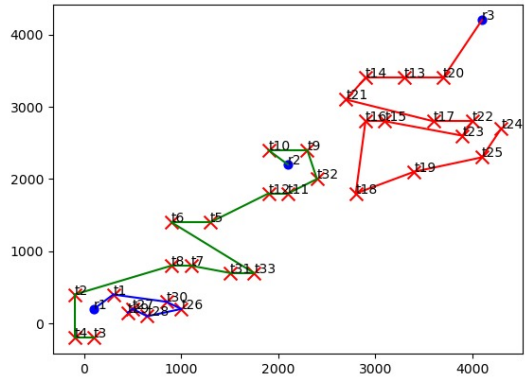

Fig. 18: Visualization of 3 robots and 25 tasks

shown a strong space discovering for small number of tasks less than 50 task in a short time. In addition,the algorithms conclude that they have a strong space search capability except SA(not population based), so that they can make the robot map reach the task fast, and obtain the global optimal scheme for the task allocation problem. However, with increasing the number of robots and tasks in the environment, the four proposed algorithms become more sluggish to find the optimal or sub-optimal solution.in addition SA was the fastest algorithm to converge but it get trapped easily into the local

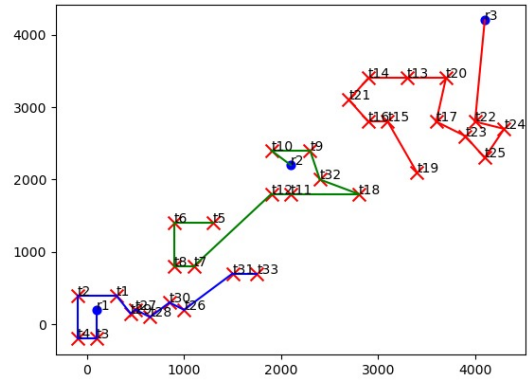

Fig. 19: Visualization of 3 robots and 25 tasks

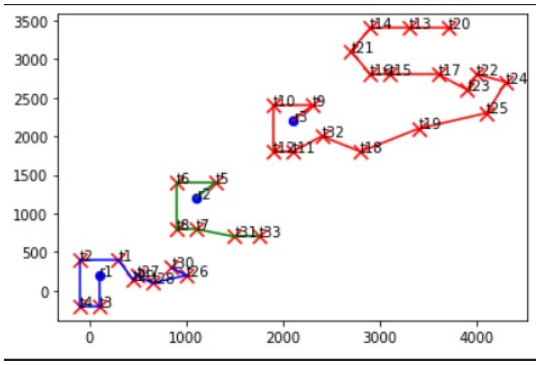

Fig. 20: Visualization of 4 robots and 33 tasks

Randomly initialize the whale population.
Evaluate the fitness values of whales and find out the best search agent $\boldsymbol{X}^*$.
while $t < t_{max}$
  Calculate the value of a according to Equation (13)
  for each search agent
    if $h < 0.5$ then
      if $|\boldsymbol{A}| < 1$ then $\boldsymbol{X}(t+1) = \boldsymbol{X}^*(t) - \boldsymbol{A} \cdot \boldsymbol{D}$
      else if $|\boldsymbol{A}| \geq 1$ then $\boldsymbol{X}(t+1) = \boldsymbol{X}_{rand}(t) - \boldsymbol{A} \cdot \boldsymbol{D}'$
      end if
    else if $h \geq 0.5$ then
      $\boldsymbol{X}(t+1) = \boldsymbol{D}' \cdot e^{bl} \cdot \cos(2\pi l) + \boldsymbol{X}^*(t)$
    end if
  end for
  Evaluate the fitness of $\boldsymbol{X}(t+1)$ and update $\boldsymbol{X}^*$
end while

Fig. 21: Whale Algorithm Pseudo Code

optimum solution ,on the other hand both genetic and ACO shows a strong exploration rate in comparison with SA.as the exploration rate for both genetic and ACO is much more than SA as both genetic and ACO are population based.in case of whale which is suitable for the continuous problem we have used a hybrid genetic with whale to make it suitable for the discrete problems from the results it show the effective of this modification as it added an additional exploration power for the algorithm.

Begin
  Initialization. Set the generation counter $t = 1$; Initialize the population of NP bats P randomly and each bat corresponding to a potential solution to the given problems; define loudnes A_i, pulse frequency Q_i and the initial velocities v_i $(i = 1, 2, \ldots, NP)$; set pulse rate r_i.
  While the termination criterion is not satisfied ***or*** $t < MaxGeneration$ ***do***
    Generate new solutions by adjusting frequency, and updating velocities and location Solutions (2),
    if (rand $> r_i$) **then**
      Select a solution among the best solutions;
      Generate a location solution around the selected best solution
    endif
    Generate a new solution by flying randomly
    if (rand $< A_i$ && $f(x_i) < f(x_*)$)
      Accept the new solution
      Increase r_i and reduce A_i
    endif
    Rank the bats and the find the current best x_*
    $t = t + 1$;
  endwhile
    Post-processing the results and visualization.
end.

Fig. 22: The bat algorithm pseudo code

REFERENCES

[1] S. Y. N. Y. E. I. Tkach, A. Jevti'c, "A modified distributed bees algorithm for multi-sensor task allocation," *In Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics (SMC), Manchester, UK, 13–16 October 2013; pp. 1401–1406. Received: 15 December 2017; Accepted: 27 February 2018; Published: 2 March 2018.*

[2]

[3] J. Y. Z. Zhu, B. Tang, "Multirobot task allocation based on an improved particle swarm optimization approach," *International Journal of Advanced Robotic Systems Date Received: 11 September 2016; accepted: 26 April 2017.*

[4] A. A. H. Qizilbash, C. Henkel, and S. Mostaghim, "Ant colony optimization based multi-robot planner for combined task allocation and path finding," in *2020 17th International Conference on Ubiquitous Robots (UR)*, 2020, pp. 487–493.

[5] Chen Jianping, Yang Yimin, and Wu Yunbiao, "Multi-robot task allocation based on robotic utility value and genetic algorithm," in *2009 IEEE International Conference on Intelligent Computing and Intelligent Systems*, vol. 2, 2009, pp. 256–260.

[6] Lishan Kang, Aimin Zhou, B. McKay, Yan Li, and Zhuo Kang, "Benchmarking algorithms for dynamic travelling salesman problems," in *Proceedings of the 2004 Congress on Evolutionary Computation (IEEE Cat. No.04TH8753)*, vol. 2, 2004, pp. 1286–1292 Vol.2.